

Chapter 2

Fundamental concepts

When I let go of what I am,
I become what I might be.

Modern Taoist interpretation of
a potential saying by Laozi, but
no ancient Chinese version exists
in the Tao Te Ching ([Laozi, 2009](#)).

We live in an era of multi-messenger astronomy, meaning that various instruments operational in a wide range of the electromagnetic spectrum cooperate to collect data. In computational astrophysics we have the concepts of multi-scale and multi-physics modelling, which are the equivalents of multi-messenger astronomy. Here, we discuss the fundamental aspects that enables multi-messenger computational astrophysics. Fundamental parts include data handling, communication, unit conversion, and scaling.

2.1 Programming

The majority of code development you will do as a apprentice AMUSE user will be in the Python programming language ([van Rossum, 1995a,b,c,d](#)). In the previous version of this book, we adopted Python version 2.7, this version of the book is written with Python 3.7+ in mind ([Van Rossum & Drake, 2009](#)).

2.1.1 Why Python

Different programming languages, like Fortran ([American National Standards Institute, 1978](#)), C ([Kernighan & Ritchie, 1988](#)), and Python have been the main language of use for us. The first two authors moved to C++ ([Stroustrup, 2013](#)) to write an extensive N-body simulation environment called Starlab ([Hut *et al.*, 2010](#)).

Many of the modules in AMUSE are written in these languages. Although our preferences differ, and we may nowadays prefer C++, Julia ([Bezanson *et al.*, 2017](#)), or Rust ([Steve Klabnik, Carol Nichols, 2023](#)), the main AMUSE package is written in Python.

Python has become one of the most popular and versatile programming languages, it is ease of use, versatility, reasonably efficient, data focused, and it has a great and active development community. In addition, it makes working with unstructured text almost fun, whereas this has always been one of the older languages' weaker points.

Our choice of Python gives AMUSE users access to a powerful, flexible, and widely used language that opens doors to numerous career opportunities and enables them to address a wide range of programming challenges.

For clarity, we briefly review some of the most important style recommendations when you write or develop any scripts on your own. We present these small guidelines to ensures uniform formatting across the code you write. Homogeneous formatting reduces decision making so that you can focus on solving problems. It facilitates teamwork, and improves improves maintainability through adherence to best practices.

2.1.2 Style guide

Here, we cover the most important aspects of the Python Enhancement Proposal (PEP 8, [van Rossum *et al.*, 2001](#)). Some further style recommendations are available in [Warsaw \(2022\)](#), which we recommend reading. The Python style guide includes recommendations for indentation, imports, the use of whitespace, naming conventions, comments, docstrings, and basic syntax preferences. Keep in mind though that these are just guidelines. When your specific requirement needs you to abandon them, abandon them.

Resist the urge to put in more than one import per line. But group them in order of the standard Python libraries, third party libraries, and as last your local sources.

```
import os
import numpy as np
from amuse.units import units
```

The first two lines in this snippet should look familiar. The third line directly uses a specific module from the AMUSE framework. This is a module that allows us to convert units from one set to another. The unit module is quite handy and general, and AMUSE was the first astronomical code that incorporated a general unit module. We will go in some depth in explaining the unit module in [Section 2.3](#).

Parameters in your script can be provided with units using the 'pipe' operator, for example a parameter `mass` can be provided with the mass of 1 solar mass by

```
from amuse.units import units
mass = 1 | units.MSun
```

Avoid the use of additional whitespace between braces, brackets and parenthesis. Please write

```
sun_and_jupiter = Particles(name=["Sun", "Jupiter"], m=(1, 0.001) | units.MSun)
```

rather than this **bad example**:

```
SunAndJupiter = Particles ( name = [ "Sun", "Jupiter" ], m = ( 1, 0.001 ) | units.
    MSun)
```

For functions, variables and methods, we use *snake_case*: lower-case separated by an underscore, and keep lines under 80 characters. Note that, in accordance with PEP 8, we prefer white spaces after commas (',') and double columns (':'), while all operators are surrounded by white space, such as in

```
mass_total = (1 | units.MSun) + (1 | units.MJupiter)
```

except for equal signs ('=') in function calls.

For classes we use *PascalCase*, where all words begin with a capital letter

```
class MilkyWayGalaxy():
```

whereas constants are all in capitals with underscores separating the words

```
# This is the distance from the Sun to the Galactic center.
SUN_GALACTIC_DISTANCE = 8.5 | units.kpc
```

Comments in a code are complete sentences. But at the start of a function we use docstrings

```
import numpy as np
from amuse.units import constants

def semi_to_orbital_period(semi_major_axis, mass_total):
    """
    Convert semi-major axis to orbital period
    """
    return 2 * np.pi * (semi_major_axis**3 / (constants.G * mass_total)).sqrt()
```

We use 4 spaces per indentation level. Note that portable document format (pdf) does not support whitespace, and those are not properly copy-pasted from the book to your Python environment.

For comparisons we use 'is' for singletons, and '==' for values

```
if x is None:
    pass
```

We regularly include Python snippets in the text. Sometimes they are written as standalone sentences or comments, and sometimes they are taken directly from one of the larger scripts in the example package. In the latter case, the code lines are associated with line numbers. These line numbers refer directly to the appropriate example script (although these could change if and when the examples are revised).

2.2 Internal data handling and communication

Data is a collection of facts in the form of bits of information, often expressed in numbers, text, images, or sound. It represents information that can be quantified and used to understand patterns, trends, and relationships in the world around us. It serves as the foundation for decision-making and leads to deeper insight. Data can be processed, manipulated, and analyzed to extract insights. As the amount and complexity of data, in science and beyond, continues to grow (probably exponentially), the need for improved data handling increases. We have no ready answer to the complexities of data analysis; we consider its handling a science, not an art.

2.2.1 Particles and Particle sets

In our experience, most of the syntactic and mathematical features of Python (conditionals, loops, arithmetic operations, etc.—even the rigid use of indentation to define program structure)—are broadly intelligible to most programmers, regardless of background. However, some more complex data structures may be unfamiliar to readers used to less structured programming languages, such as C or Fortran. Accordingly, we pause here to elaborate on this topic in AMUSE, using some important data structures as templates.

A Python class is a data structure together with a collection of associated functions, or methods, designed to manipulate the data. A particularly simple AMUSE class is a `Particle`, which contains only basic particle attributes, such as mass, position, velocity, etc., as well as a randomly generated key that uniquely identifies the particle in the AMUSE system. We can access its contents directly, and expand it to store all sorts of useful information.

```
from amuse.datamodel import Particle
from amuse.units import units

p = Particle()
p.mass = 1.0 | units.MSun
p.mass2 = 42 | units.kg
p.name = "Christine Chapel"
p.rank = "Nurse"
print(p)
```

The `particle` class is not very useful in and of itself, but it forms the basis for a vitally important class called a `particle set`, which is basically an array of particle objects, with conventions regarding content and a collection of methods providing information on and control over the data. Particle sets are used to transfer data from one code to another, and for eventual data processing and analysis. An element of a `particle set` is expected to have certain attributes, such as `mass`, `position`, `velocity`, `radius`, and possibly others. The built-in methods assume that these data exist.

You can generate a (rather useless, for now) particle set by using the following lines

```
from amuse.datamodel import Particles

particles = Particles(number_of_particles)
print(particles)
```

This prints the most fundamental ingredient in a particle set; the **key**, which identifies each particle. Note here the difference between the singular and the plural forms for declaring a particle and a particle set.

```
          key
          -
=====
5512314219721684995
8353633730176500107
9653245888285946456
9457276936209163492
14505666692805646583
14003077150553218257
=====
```

There is not much one can do with such a simple bunch of particles, except perhaps to use it as a very expensive random number generator. However, we can easily extend it in useful ways. We can create a particle set of 10 particles as follows:

```
from amuse.datamodel import Particles

particles = Particles(10)
```

We can then set up attributes in various ways:

```
particles.mass = 1 | units.MSun
```

sets all masses, while

```
particles[3].mass = 42 | units.kg
```

sets just the mass for the fourth particle (Python starts counting at 0). We could also have set the masses (or positions, etc.) in the **Particles** creation call, with

```
particles = Particles(10, mass=1 | units.MSun)
```

Some more useful ways of setting particle set attributes are presented below.

In some cases, one would like to expand the attribute list of a particle with an array. This can happen when one would like to add information about a chemical network to a particle, or some other attributes that normally is associated with particle data. Adding abundances, for example, for Hydrogen, Helium, Lithium and Iron could be realized as follows

```
particles.add_vector_attribute("abundance", ("H", "He", "Li", "Fe"))
particles.abundance = [0.75, 0.24, 0.005, 0.005]
```

In the first line we add a vector attribute called “abundance” to the particle set, and

specify the array of names for this attribute. In the second line we initialize the values for all the 10 particles with the same chemical composition.

Once the basic attributes are set, the methods associated with the `particle set` provide commonly used functionality. For example,

```
energy_kin = particles.kinetic_energy()
```

computes the total kinetic energy of the particle set, while

```
energy_pot = particles.potential_energy()
```

computes the total potential energy. Similarly,

```
com = particles.center_of_mass()
```

calculates the center of mass of the particle set, while

```
particles.scale_to_standard()
```

places the set in the center of mass frame and scales it to a standard configuration (see Section 4.1.2.1). Many AMUSE functions take a particle set as input and return analytical information on it.

You can ask Python to list the methods associated with a class by using the `dir` function:

```
dir(Particles)
```

will provide a list of all the methods available in the `Particles` class. (Note the Python convention that names beginning and ending with double underscores are intended for private use, even though you can see them and in principle use them yourself—not recommended!) More extended information can be obtained with

```
help(Particles)
```

which provides a detailed description of the methods in the class.

Even here, though, Python asserts its inherent flexibility. Unlike classes in more strongly typed languages, Python classes are customizable. You can add your own variables at will for your own purposes—just don't expect them to be properly managed or preserved by the built-in methods! Such additional attributes are also ignored by the specific modules, and will not be copied back and forth between module and local script. Just keep in mind that they have to be handled explicitly by the script writer.

2.2.2 Grid-based data

Technically grids are conceptually quite different than particles, but in principle they can represent the same information, at least numerically. Grids are also used in gravity codes (such as in tree codes (Barnes & Hut, 1986) and particle-mesh solvers (Hockney & Eastwood, 1988)), and they are quite common for hydrodynamics solvers. In principle the same information can be conveyed with particles, or with grids, but for a wide variety of reasons grids often turn out more complex because they are not necessarily

Cartesian.

Voronoi grids, for example, represent a tessellated space, whereas a KDTree-type grid or a refined grid have different data structures. These differences in the fundamental data structure makes it difficult to unify the grid-based data structures. We therefore project grids onto the already existing particle data structure.

Examples of grid-based codes include Athena and FLASH. Both use adaptive grids, although, this is not how we initialize them. These grids are most easily initialized using a Cartesian grid, after which the code internally readjusts the grid structure to reflect the properties of the gas (often the density or temperature structure).

Grid-based hydrodynamical solvers or radiative transfer codes often handle the data locally in the particular code, rather than in the AMUSE script. An example is the `Torch` script [Wall *et al.* \(2020a,b\)](#), which combines the grid-based magnetohydrodynamics solver `FLASH` with AMUSE. The grid-based data in `Flash` is handled, saved, and stored in the code. The AMUSE script can in principle touch the data, but this leads to complex data consistency responsibility issues, performance loss, and unnecessary data replication.

A simple grid for the hydro-code Athena can be initialized as follows

```
from amuse.units import generic_unit_system
from amuse.community.athena import Athena

instance = Athena()
instance.parameters.x_boundary_conditions = ("periodic", "periodic")
instance.parameters.mesh_length = (8, 1, 1) | generic_unit_system.length
instance.parameters.mesh_size = (8, 1, 1)
grid = instance.get_extended_grid()
instance.initialize_grid()
print(grid.rho)
```

A more generic way of making a grid is

```
def make_grid(number_of_grid_cells,
              length,
              constant_hydrogen_density,
              inner_radius,
              outer_radius
              ):
    grid = Grid.create([number_of_grid_cells] * 3,
                      length.as_vector_with_length(3)
                      )
    grid.radius = grid.position.lengths()
    grid.hydrogen_density = constant_hydrogen_density
    grid.hydrogen_density[grid.radius <= inner_radius] = 0 | units.cm ** -3
    grid.hydrogen_density[grid.radius >= outer_radius] = 0 | units.cm ** -3
    return grid
```

We can plot the particles of a gaseous distribution simply using matplotlib, as demonstrated in [Figure 2.1](#).

In [Figure 2.2](#) we show three representations of a gaseous distribution (right), as a Voronoi tessellated structure (left) and as an adaptive Cartesian grid (middle).

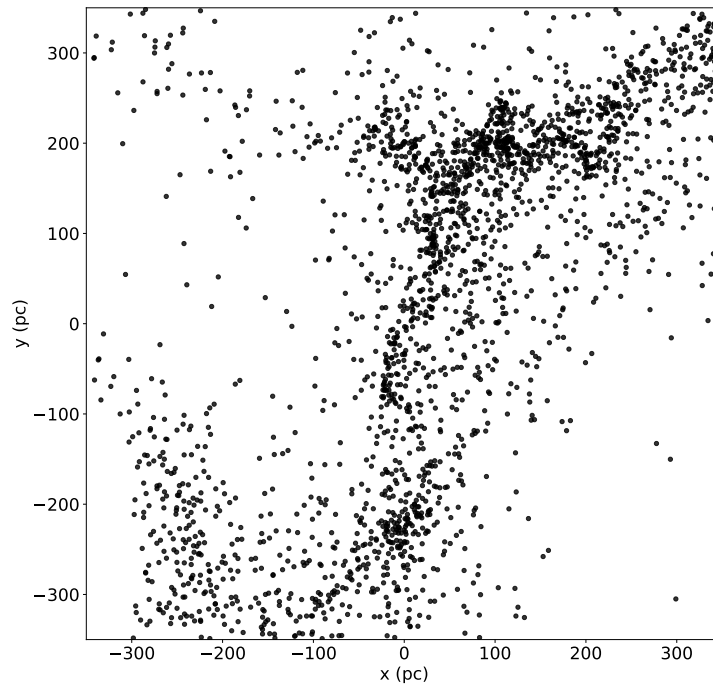


Figure 2.1: The calculation was performed using the Torch package in AMUSE. The projection is 700×700 pc. Tessellated, Adaptive grid and the gaseous distributions are presented in Figure 2.2. The script to generate this figure can be found in `ae.hydro.voramr_logic_progression_plots`.

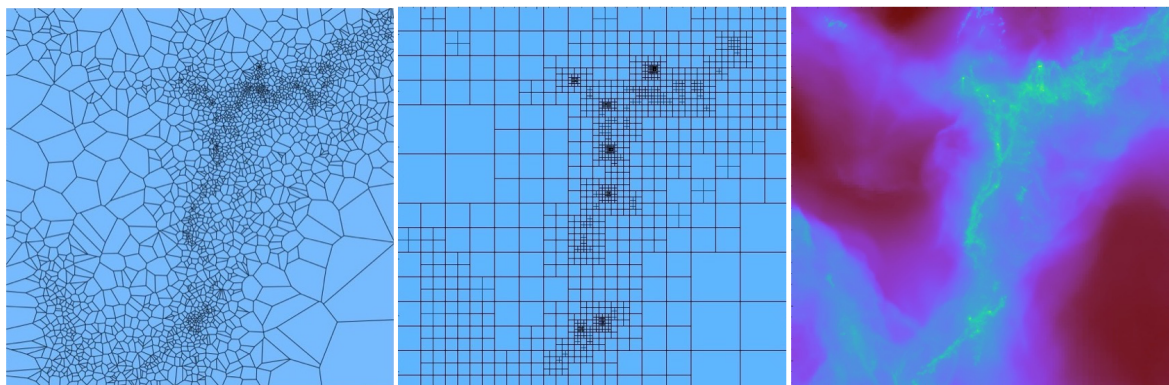


Figure 2.2: Voronoi tessellation (left) (Lewis *et al.*, 2023), adaptive Cartesian grid (middle) and a gaseous representation (right) from a simulation performed using FLASH. The projection is 700×700 pc. in Figure 2.1 we show the particle gas distribution of the same filament. The script to generate this figure can be found in `ae.hydro.voramr_logic_progression_plots`.

We construct and plot the Voronoi tessellation from the particle gas `particle_data`, using the `scipy` package `Voronoi`.


```
from scipy.spatial import Voronoi, voronoi_plot_2d
voronoi_data = Voronoi(
    particle_data, furthest_site=False, qhull_options="Qbb Qc Qz Gc",
)
voronoi_plot_2d(voronoi_data, ax, show_vertices=False)
```

The refined grid is plotted conveniently using the YT package (see <https://yt-project.org/doc/index.html>).

```
AMR_data = yt.load("../data/lv112-VorAMR_hdf5_chk_0001")
plt = yt.ProjectionPlot(AMR_data, "z", "bgpt")
plt.set_cmap(field=("flash", "bgpt"), cmap="arbre")
plt.annotate_grids(cmap="black_only")
```

Several codes make use of grids. Some of them solve for gravity, such as the various tree codes, and others solve for either hydrodynamics or radiative transfer. We will have more to say on these codes in their respective chapters: Chapters 4, 6 and 7.

2.2.3 Data handling between code and script

Let's imagine that particle information (mass, position, velocity, etc.) is stored in a particle set called `bodies`, which is part of local memory and lives in the user Python script. This data does not naturally live in the selected worker code, but the essential parts can be copied to the code as needed. For a stellar evolution code this might be the mass and radius, and for an N-body code this could also include position and velocity. This local information is not automatically updated after the code has been run using `{code}.evolve_model()`. Here `{code}` may be the instantiation of any type of code for evolving stars or integrating equations of motion.

To access the new data, we could copy it directly from the remote code by referencing `{code}.particles`, which lives in the code interface and is updated by `{code}.evolve_model()`, but each access will cause AMUSE to open a one-time MPI or sockets connection to the running code where the remote data live, which can be slow in terms of compute time because it requires the construction of the connection. More importantly, it is less explicit. Note that some codes may have various types of particles. This is not uncommon in particle-based hydrodynamics solvers, which may make a distinction between `dm_particles` for dark matter, and `gas_particles` for the gas.

A more efficient way to communicate with a running code is realized by creating a *channel* to the remote data of the running worker code. A communication channel for (in this case) a gravity code is declared with

```
channel_from_gravity_to_framework = gravity.particles.new_channel_to(bodies)
```

Here `gravity` is a pointer to the running code. To copy the particle data from the running code to the local Python framework, we say

```
channel_from_gravity_to_framework.copy()
```

Now the masses, positions, and velocities of the remote data in the worker module are duplicated in the local `bodies` particle set in the framework (Python) script. The local data (in `bodies`) and the remote data (in `gravity.particles`) are consistent. The `copy()` command copies all data, but to tune performance or to prevent overwriting some local data, one can choose to copy only part of the data. For example, to update only the positions, but not the masses, velocities, or other attributes, we could write:

```
channel_from_gravity_to_framework.copy_attribute(["x", "y", "z"])
```

We could have arranged for the local data to be automatically synchronized with the worker data on each return from the worker code, but for the sake of user control and efficiency, we have opted instead for manual synchronization.

2.2.4 Storing and Recovering Data

We can store particle data in a file for further and future analysis. It is advisable to synchronize the local `bodies` with the code data (see Section 2.2.3) before writing them to a file. A snapshot can be saved to a file named `filename` as follows:

```
write_set_to_file(bodies, filename)
```

Optionally, a “`format={format}`” argument indicates the format in which the snapshot is stored—by default “`amuse`”, which uses `hdf5`. This format is commonly used within the astronomical community; it provides a self-descriptive binary format with the potential of storing hierarchical data structures ([The HDF Group, 2000-2010](#)). Other recognized types include “`starlab`” (the internal StarLab `dyn` and `star` formats [Portegies Zwart *et al.* \(1999\)](#); [Hut *et al.* \(2010\)](#)), “`csv`” (comma-separated-values), and plain text (“`txt`”).

Time is not an attribute of the particle set, but rather of the code, in this case the N -body integrator. As a consequence, the age of a particle is not stored by default. This generally saves data and the historical context stems from the idea that all stars are born at the same time. Some codes do not even allow for particles to have different ages, but in AMUSE we are not bound by these limitations. To save the time of a snapshot, we can use:

```
write_set_to_file(bodies.savepoint(0 | model_time.unit), filename)
```

Here, the phrase `bodies.savepoint` ensures that the time is stored with the snapshot. A stored output file can be read back to memory:

```
bodies = read_set_from_file(filename)
```

If time was saved with the snapshot, it can be recovered:

```
model_time = bodies.get_timestamp()
```

For storing the ages of individual particles, one can simply add a new attribute to the particle set:

```
bodies.age = 0 | units.Myr
```

or to an individual particle:

```
Sun.age = 4.5678 | units.Gyr
```

Multiple snapshots can be stored in a single file. In that case, one can iterate over the individual snapshots using

```
many_snapshots = read_set_from_file(filename)
for snapshot in list(many_snapshots.history):
    print(f"The number of bodies in this snapshot is: {len(snapshot)}")
```

For runs with a large number of particles, or containing many snapshots, these files can become quite large. One solution is to split the output into smaller pieces and process them separately. Sometimes, however, this is not desirable. In such cases, one can keep the data on disk by keeping the file open while reading:

```
bodies = read_set_from_file(
    filename,
    copy_history=False,
    close_file=False,
)
```

Alternatively, one could write compound particle sets to a single file and recover them with:

```
gas, stars = read_set_from_file(
    filename,
    names=["gas", "stars"],
    copy_history=False,
    close_file=False,
)
```

Yet another way, which is often used in planetary systems, is to make a distinction in the particle set by introducing an extra attribute:

```
stars.name = "star"
planets.name = "planet"
moons.name = "moon"
bodies = stars + planets + moons
write_set_to_file(filename, bodies)
```

After reading the file one can split the particle set:

```
bodies = read_set_from_file(filename)
stars = bodies[bodies.name == "star"]
```

and similarly for the other particle subsets.

2.3 Converting between code units and SI

2.3.1 Scaling standard units to internal code units

Units are fully supported in AMUSE. Running one of the interfaced high-performance codes with units is not difficult. For intrinsic dimensionless solvers including units results in the loss of the scale-free characteristics of the problem. This is only a problem in gravitational dynamics, which can be addressed elegantly as a scale-free problem using Hénon units (Hénon, 1971). For stellar evolution, unit conversion is generally not an issue because both the Henyey and the parameterized stellar evolution codes already work in units that naturally convert to SI. Similar arguments hold for hydrodynamics solvers and radiative transfer codes, but they tend to be written for specific sets of units. We desire to achieve coherency and consistency in the use of units, and therefore introduce the converter class. If you are unfamiliar with Python classes, don't despair, as we provide a quick overview in Appendix C.

The units in an N -body system are specified by providing any two of the mass, length, and time scales of the problem. As a practical matter, conversion to other systems requires the creation of a `converter`, which is a Python class specifically designed to keep track of units. We can construct a converter with mass unit $1 M_{\odot}$ and spatial unit 1 au as follows:

```
converter = nbbody_system.nbbody_to_si(1 | units.MSun, 1 | units.au)
```

However, the following line

```
converter = nbbody_system.nbbody_to_si(72 | units.kg, 175 | units.cm)
```

which happens to be the weight and height would work equally well. In this last example we really did use the International System of Units, whereas for most astronomical simulations, we are more likely to use solar masses, millions of years, and parsecs. Nevertheless, the conversion system is still generically called `nbbody_to_si`.

A system in physical units can be converted back to N -body units using the same converter. For example, converting the positions back to N -body units can be accomplished with

```
positions_in_nbbody_units = converter.to_nbbody(bodies.position)
```

or back to SI units:

```
positions_in_si_units = converter.to_si(positions_in_nbbody_units)
```

More information about alternative converters can be found in Section 2.3.2

2.3.2 Alternative converters

Instead of using a mass and a time, or a mass and a length-scale, to specify the set of units it is also possible to use instead luminosity and density, or any other dimensionally independent quantities. The routine for specifying a different set of units can be imported by

```
from amuse.units.generic_unit_converter import ConvertBetweenGenericAndSiUnits
```

We can call this routine with (for example) luminosity `L` and density `rho`, would look as follows:

```
converter = ConvertBetweenGenericAndSiUnits(L, rho, constants.G)
```

Alternatively, it can be called with any set of independent units:

```
converter = ConvertBetweenGenericAndSiUnits(
    1.0 | units.kpc,
    1.0e10 | units.MSun,
    constants.G,
)
```

or if `Rvir` is virial radius and `rho` is density,

```
converter = ConvertBetweenGenericAndSiUnits(Rvir, rho, constants.G)
```

2.3.3 Changing Parameters of the simulation

Often the functionality of a community code needs to be modified, for example to improve accuracy or set some other parameter. This is controlled by internal parameters within the code. Changing those internal parameters is always possible, but we want to avoid modifying and recompiling the code. For this reason, many internal code parameters can be queried and (re)set. The names of these parameters may be the same for some codes, but completely different for others. Fortunately, as described in Section 2.2.1, Python and the AMUSE interface have the ability to provide help on these matters. The command

```
dir(Ph4)
```

provides a extensive list of all the ingredients of the direct N -body code `Ph4` (see Chapter 4 for a more detailed description of N -body codes). This command can be given for each code, such as

```
dir(Mocassin)
```

The same command with the code `Bhtree` will provide a different list of functionalities for the tree code, and equally wise for the radiative transfer code `Mocassin`. More extended information can be obtained with

```
help(Ph4)
```

A list of parameters for `Ph4`, and their default values, can be obtained for the instantiation of a particular code, such as for the same gravity solver

```
gravity = Ph4()
```

we can get information about the parameter settings through

```
gravity.parameters?
```

A list of all available parameters (and their default settings) of this particular code is provided by

```
print(gravity.parameters)
```

but the same constructs such as `help` and `dir` can also be applied to the instantiation of the code, or to its member functions.

The parameters used in two seemingly similar codes may have different names. This sounds very confusing, but it reflects the diversity of the authors of the various interfaces. Our philosophy here is to try to reserve identical names for parameters that do exactly the same thing, but because many codes have slightly different functions or objectives, it is hard to be consistent and clairvoyant at the same time. In Section 4.3.6

Some parameters control the actual working of a code. A smoothed-particles hydrodynamics code, such as `Fi()` allows the user to control the number of nearest neighboring particles to determine the local thermodynamical parameters. This can be tuned by setting the parameter from the default value of 32 to, one can write:

```
hydro.parameters.targetnn = 64
```

The number of free parameters varies a lot depending on the code. The stellar evolution code `Seba()` has 11 parameters, the `Ph4()` *N*-body code has 20, and the smoothed-particles hydrodynamics code `Fi()` has 60. Having more (or fewer) parameter does not necessarily say much about the reliability of the code, but it does say something about the effort we took in making these parameters accessible by the user from the Python interface.

The best way to learn about the various parameter settings, and the possibilities of a specific code is by looking at good old-fashioned source code, as all parameter settings are defined in the interface file of each community module in the AMUSE source code. For the `Hermite` package, for example, the parameters can be found in the method `define_parameters()` in the `amuse.community.hermite.interface.Hermite` class.

2.4 Tricks with units

2.4.1 Setting printing strategies

Printing quantities with units can be awkward, because the units often come out in undesired form: $1M_{\odot}$ might be printed in grams, or 1 pc in centimeters. To prevent this unwanted behavior, you could explicitly convert to your desired units, e.g.

```
x = 10 | units.cm
print(f"x={x.in_(units.parsec)}")
```

but this is quite cumbersome and inconvenient for routine use. An alternative is to set a preferred printing strategy. For example, if your script performs all calculations in

solar masses, solar radii, and megayears, you can indicate your preferred set of units with

```
set_printing_strategy(
    "custom",
    preferred_units=[units.MSun, units.RSun, units.Myr],
    precision=6,
    prefix="",
    separator=" [",
    suffix="]",
)
```

This will ensure that all numbers will be printed with 6 decimal places and with the units in brackets. But when you perform a galaxy collision simulation you may prefer

```
set_printing_strategy(
    "custom",
    preferred_units=[units.MSun, units.kpc, units.Gyr]
)
```

To print in other unit systems, such as N-body units, a converter must be provided in the argument list, for example, `nbody_converter = converter` (see Section 2.3 for details).

2.4.2 The command-line argument parser

Production scripts usually require tuning for application to a specific problem, and it is desirable to be able to change a script's input parameters, such as the number of stars, their masses or metallicity, the virial radius of a cluster, or the position of a black hole, without having to edit it every time. These parameters are traditionally set on the command line and subsequently parsed by the production script.

One solution in Python for managing command-line arguments is the argument parser

```
import argparse
```

Code example 2.1 presents a usage example of this command-line argument parser, where we allow the stellar mass `M`, metallicity `z`, and the end time of the simulation `model_time` to have the desired units.

Bibliography

- American National Standards Institute. 1978. *American National Standard Programming Language FORTRAN*. Technical Report ANSI X3.9-1978. American National Standards Institute, New York, NY.
- Barnes, Josh, & Hut, Piet. 1986. A hierarchical $O(N \log N)$ force-calculation algorithm. *Nat*, **324**(6096), 446–449.
- Bezanson, Jeff, Edelman, Alan, Karpinski, Stefan, & Shah, Viral B. 2017. Julia: A fresh approach to numerical computing. *SIAM review*, **59**(1), 65–98.


```

50 def new_argument_parser():
51     parser = argparse.ArgumentParser(
52         formatter_class=argparse.ArgumentDefaultsHelpFormatter,
53     )
54     parser.add_argument(
55         "-m",
56         type=units.MSun,
57         dest="mass",
58         default=1.0 | units.MSun,
59         help="stellar mass",
60     )
61     parser.add_argument(
62         "-t",
63         type=units.Myr,
64         dest="time_end",
65         default=4700.0 | units.Myr,
66         help="end time of the simulation",
67     )
68     parser.add_argument(
69         "-z",
70         dest="metallicity",
71         type=float,
72         default=0.02,
73         help="metallicity",
74     )
75     return parser
76
77
78 def main(**kwargs):
79     stellar_minimal(**kwargs)
80
81
82 if __name__ == "__main__":
83     arguments = new_argument_parser().parse_args()
84     main(**arguments.__dict__)

```

Code example 2.1: Command-line parser for a stellar evolution model and the main routine for calling the function listed in Code example 5.3 in Section 5.2.2. This snippet is taken from `amuse.examples.stellar_minimal`.

- Hénon, M. H. 1971. The Monte Carlo Method (Papers appear in the Proceedings of IAU Colloquium No. 10 Gravitational N-Body Problem (ed. by Myron Lecar), R. Reidel Publ. Co. , Dordrecht-Holland.). *Ap&SS* , **14**(1), 151–167.
- Hockney, R.W., & Eastwood, J.W. 1988. *Computer Simulation Using Particles*. Adan Hilger Ltd., Bristol, UK, 540 pp.
- Hut, Piet, McMillan, Steve, Makino, Jun, & Portegies Zwart, Simon. 2010 (Oct.). *Starlab: A Software Environment for Collisional Stellar Dynamics*. Astrophysics Source Code Library, record ascl:1010.076.
- Kernighan, Brian W., & Ritchie, Dennis M. 1988. *The C Programming Language*. 2 edn. Englewood Cliffs, NJ: Prentice Hall.
- Laozi. 2009. *Tao Te Ching*. Dao de jing. English. London: Frances Lincoln Limited. Translated from Chinese. Subtitle on cover: "An illustrated journey".
- Lewis, Sean, McMillan, Stephen, Mac Low, Mordecai-Mark, Cournoyer-Cloutier,

- Claude, Polak, Brooke, Tran, Aaron, Wilhelm, Maite, Sills, Alison, Klessen, Ralf, & Wall, Joshua. 2023 (Jan.). Star Cluster Formation: The effects of early forming massive stars and building a bridge between Voronoi mesh and block-structured codes. *Page 109.03D of: American Astronomical Society Meeting Abstracts #241*. American Astronomical Society Meeting Abstracts, vol. 241.
- Portegies Zwart, S. F., Makino, J., McMillan, S. L. W., & Hut, P. 1999. Star cluster ecology. III. Runaway collisions in young compact star clusters. *A&A* , **348**(Aug.), 117–126.
- Steve Klabnik, Carol Nichols. 2023. *The Rust Programming Language*.
- Stroustrup, Bjarne. 2013. *The C++ Programming Language*. 4 edn. Upper Saddle River, NJ: Addison-Wesley.
- The HDF Group. 2000-2010. *Hierarchical data format version 5*.
- van Rossum, Guido. 1995a (Apr.). *Extending and embedding the Python interpreter*. Report CS-R9527. pub-CWI, pub-CWI:adr.
- van Rossum, Guido. 1995b (Apr.). *Python library reference*. Report CS-R9524. pub-CWI, pub-CWI:adr.
- van Rossum, Guido. 1995c (Apr.). *Python reference manual*. Report CS-R9525.
- van Rossum, Guido. 1995d (Apr.). *Python tutorial*. Report CS-R9526.
- Van Rossum, Guido, & Drake, Fred L. 2009. *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace.
- van Rossum, Guido, Warsaw, Barry, & Coghlan, Nick. 2001 (jul). *PEP 8 – Style Guide for Python Code*. Python Enhancement Proposal 8. Python Software Foundation. Last updated: 2013-08-01.
- Wall, Joshua E., Mac Low, Mordecai-Mark, McMillan, Stephen L. W., Klessen, Ralf S., Portegies Zwart, Simon, & Pellegrino, Andrew. 2020a. Modeling of the Effects of Stellar Feedback during Star Cluster Formation Using a Hybrid Gas and N-Body Method. *ApJ* , **904**(2), 192.
- Wall, Joshua E., Tran, Aaron, Mac Low, Mordecai-Mark, McMillan, Stephen L. W., Klessen, Ralf S., Portegies Zwart, Simon, & Pellegrino, Andrew. 2020b (Mar.). *Torch: Coupled gas and N-body dynamics simulator*. Astrophysics Source Code Library, record ascl:2003.014.
- Warsaw, Barry A. 2022. *GNU Mailman Coding Style Guide*. Accessed: [Insert Date].